

P2636R1

References to ranges should always be viewable

Published Proposal, 2022-10-14

Author:

[Hannes Hauswedell](#)

Audience:

LEWG, SG9

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that the `views::all` adaptor and the `viewable_range` concept accept lvalue references to all ranges (including move-only views).

Table of Contents

1 Revision History

1.1 Revision 1 - 2022-10-14

1.2 Revision 0 - 2022-09-14

2 Introduction

2.1 The problem (TL;DR)

2.2 The solution (TL;DR)

3 Detailed examples

3.1 Views from lvalue reference

3.2 View composition (starting from lvalue reference)

- 3.3 Views from rvalues
- 3.4 View composition (starting from rvalue)
- 3.5 Generic code

4 Proposal

- 4.1 Wording
- 4.2 Impact

5 "Workarounds"

- 5.1 `std::views::ref`

6 Lifetime

- 6.1 Returning views on local variables
- 6.2 Returning views on in-parameters

7 Implication for the "meaning" of views

8 Summary

9 Acknowledgments

10 References

§ 1. Revision History

§ 1.1. Revision 1 - 2022-10-14

- Some language corrections and smaller additions to text.
- Added "[The solution \(TL;DR\)](#)".
- Added "[Implication for the "meaning" of views](#)".
- Added section on `std::views::ref`.
- Feature test macro added to proposed wording.

§ 1.2. Revision 0 - 2022-09-14

- Initial version.

§ 2. Introduction

When dealing with views, two concepts are important: `view` ("ranges that are views themselves") and `viewable_range` ("ranges that can be turned into views"). Initially, the requirements for the first were very strict, encompassing only non-owning, semi-regular types. The second concept encompassed "views and any lvalue-references to ranges".

Over time, both concepts were relaxed considerably. Views can now be move-only ([P1456](#)) and owning ([P2415](#)), rvalues of non-views are now viewable—in fact most ranges people encounter are now viewable. However, in one particular case, the concepts are now stricter than before: lvalue-references to certain ranges, move-only views, are not viewable. This means that while almost all non-view ranges can be easily composed with view adaptors, certain views now do not qualify for this.

This issue was briefly discussed on the LWG reflector in 2020 where another member highlighted the wide-ranging impact of this change:

I'm concerned about having components that work with ranges but not views, given that views are supposed to be just ranges with additional properties.

A possible "fix" was mentioned in that discussion:

I can envision an alternate design in which `views::all(E)` does admit move-only lvalue views [...] I think that design change needs LEWG approval [...]

But apparently there was no support for this change or no will to pursue it further. This paper picks up that discussion and argues why such a change would indeed make a lot of sense.

§ 2.1. The problem (TL;DR)

```
void foobar(ranges::forward_range auto && r)
{
    auto v = r | views::take(3);
}
```

In the original C++20 standard and in multiple shipped compiler versions, the above code was well-formed for all `forward_range`s. Since P1456, the function template is now underconstrained and will become ill-formed for certain inputs.

§ 2.2. The solution (TL;DR)

Change the definition of `views::all` (and `viewable_range`) so that:

- rvalues are moved
- lvalues are copied if that is possible and cheap
- other lvalues are wrapped in `ref_view` ~~except lvalues to views~~

This means that all lvalue ranges become viewable. The only non-viewable ranges are non-movable rvalues.

§ 3. Detailed examples

To illustrate the different behaviours, I have chosen to provide examples that start out with (1) a copyable view, (2) a copyable non-view range, (3) a move-only non-view range, and (4) a move-only view.

As example types for non-copyable ranges, I use `Generator` and `GeneratorView`. `Generator` could refer to a user-provided type that generates objects upon iteration (e.g. records of data from a stream); it is a move-only single-pass input-range.

The type `GeneratorView` is the same as `Generator` except that it also inherits from the empty base class `ranges::view_base`. This has no influence on the semantics of the type other than it begin treated as a view by current standard libraries. While we might suggest to users that such ranges should always be implemented as views, we cannot assume that this will always be the case (there are ranges that are neither containers nor views!).

§ 3.1. Views from lvalue reference

(1)	<pre>string_view s{"foobar"}; auto v = s views::take(3);</pre>	s is decay_copyd
(2)	<pre>string s{"foobar"}; auto v = s views::take(3);</pre>	s is bound by ref_view
(3)	<pre>Generator g{}; auto v = g views::take(3);</pre>	g is bound by ref_view
(4)	<pre>GeneratorView g{}; auto v = g views::take(3);</pre>	currently ill-formed

Before [P1456](#) and [P2415](#), all of these snippets would have been well-formed. Currently, (4) is not, because lvalue references to move-only views are not viewable.

This also means that code is rejected by current compilers that was accepted by previous versions of the same compiler (GCC, MSVC). While this is true for other C++20 features, the particular changes to the view concepts were discussed mainly as relaxing constraints and not as introducing new ones.

§ 3.2. View composition (starting from lvalue reference)

(1)	<pre>string_view s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	s is <code>decay_copyd</code> v1 is <code>decay_copyd</code>
(2)	<pre>string s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	s is bound by <code>ref_view</code> v1 is <code>decay_copyd</code>
(3)	<pre>Generator g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	g is bound by <code>ref_view</code> v1 is <code>decay_copyd</code>
(4)	<pre>GeneratorView g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	currently ill-formed

(4) is still ill-formed; the rest are well-formed.

§ 3.3. Views from rvalues

(1)	<pre>auto v = string_view{"foobar"} views::take(3);</pre>	string_view is <code>decay_copyd</code>
(2)	<pre>auto v = string{"foobar"} views::take(3);</pre>	string is bound by <code>owning_view</code>
(3)	<pre>auto v = Generator{} views::take(3);</pre>	generator is bound by <code>owning_view</code>
(4)	<pre>auto v = GeneratorView{} views::take(3);</pre>	generator is <code>decay_copyd</code> (actually a move)

All of these snippets are currently well-formed code. In the original ranges proposal, only (1) would have been well-formed, because the others are rvalues of non-views; (4) would not have been regarded as a view, because it is move-only.

§ 3.4. View composition (starting from rvalue)

(1)	<pre>auto v1 = string_view{"foobar"} views::take(3); auto v2 = v1 views::transform(/**/);</pre>	string_view is decay_copyd v1 is decay_copyd
(2)	<pre>auto v1 = string{"foobar"} views::take(3); auto v2 = v1 views::transform(/**/);</pre>	string is bound by owning_view line 2 is currently ill-formed
(3)	<pre>auto v1 = Generator{} views::take(3); auto v2 = v1 views::transform(/**/);</pre>	generator is bound by owning_view line 2 is currently ill-formed
(4)	<pre>auto v1 = GeneratorView{} views::take(3); auto v2 = v1 views::transform(/**/);</pre>	generator is decay_copyd (actually a move) line 2 is currently ill-formed

(2)-(4) are not well-formed, because `v1` in the respective snippet is move-only. Note, how the well-formed snippets in this example are different from [those created from lvalues](#). If the `| views::transform(/**/)` had been appended to the first line and no `v2` created, it would have worked. This means that—contrary to what many previous ranges guides suggest—it makes an important difference now whether you compose the view incrementally or in one line. And it appears that "creating in one line" needs to be recommended.

§ 3.5. Generic code

Returning to the problem of generic code, assume the following:

(definitions for `to_upper` and `to_lower` omitted)

```
void print_up_low(ranges::forward_range auto && rng)
{
    for (char c : rng | views::transform(to_upper))
        cout << c;
    cout << '\n';
    for (char c : rng | views::transform(to_lower))
        cout << c;
}
```

(1a)	<pre>print_up_low(string_view{"foobar"}); print_up_low(string_view{"foobar"} views::take(3));</pre>	both calls are well- formed
(2a)	<pre>print_up_low(string{"foobar"}); print_up_low(string{"foobar"} views::take(3));</pre>	first call is well-formed second call is ill-formed
(1b)	<pre>string_view s{"foobar"}; print_up_low(s); print_up_low(s views::take(3));</pre>	both calls are well- formed
(2b)	<pre>string s{"foobar"}; print_up_low(s); print_up_low(s views::take(3));</pre>	both calls are well- formed

These examples show that it is easy to run into problems with "move-only views", even when just using simple adaptors on types like strings and string_views. Is the behaviour shown above intended? Do we expect developers to understand why 1 of 8 calls to `print_up_low()` is ill-formed and why?

Curiously, it is now the example of "creating a view in one line" that is ill-formed, while previous examples suggested that exactly this usage pattern increases the likelihood of the code being well-formed.

in practice more code will work than before and not less. In fact, this will *unbreak* code that was well-formed in earlier published versions of the IS (and is well-formed in released versions of GCC and MSVC) but that has been broken in the meantime.

Should LEWG accept this proposal for C++23, it could ask LWG to denote it as a C++20 defect fix. There is precedent for this, because the `view` and `viewable_range` concepts have already been changed multiple times with wide-ranging modifications being back-ported to C++20 and even older compiler versions (GCC10, GCC11); although technically these changes were feature changes more than fixes. The current changes are much smaller in scope and re-establish initial C++20 behaviour.

§ 5. "Workarounds"

For the previously shown ill-formed snippets, it is possible to work around the problem by any or either of the following methods:

1. `move`-ing the initial range into the pipe (resulting in an `decay_copy` or `owning_view`); or
2. manually wrapping the initial range in `ref_view`

Since most users never directly interact with `views::all`, `ref_view` and `owning_view`, I assume that it will be very hard to explain where (and which of) these workarounds are necessary.

Furthermore, this creates a burden on generic code that will either need to `if constexpr` on ranges passed to it by reference or blindly wrap them in `ref_view` to make them viewable.

Note also that "just adding `ranges::ref_view`" in a generic context may have undesired side-effects. Even for ranges that are already viewable, it is not a no-op; e.g. it prevents copyable views from being captured by value which changes their behaviour with regard to `const` (they are now shallow const but would have been deep const before). It also changes type-specific behaviour in view adaptor objects like the "same-type optimisations" in `views::take` and `views::drop`.

§ 5.1. `std::views::ref`

It was suggested that having `views::ref` as a shortcut for `ranges::ref_view` would alleviate the problems presented here. While I do not object to having such a view factory, I do not see how it solves the fundamental problem of a bad default.

It is of no consequence to almost all algorithms whether they are operating on a move-only view or a copyable range, so we shouldn't require a workaround.

§ 6. Lifetime

Previous mailing list comments suggested that the changes proposed here would create issues with view lifetime. The following two examples explore some of those implications.

§ 6.1. Returning views on local variables

Nr.	Code	Status quo	This proposal
(1)	<pre>string_view s{"foobar"}; return s views::take(3);</pre>	not dangling	not dangling
(2)	<pre>string s{"foobar"}; return s views::take(3);</pre>	dangling	dangling
(3)	<pre>Generator g{}; return g views::take(3);</pre>	dangling	dangling
(4)	<pre>GeneratorView g{}; return g views::take(3);</pre>	ill-formed	dangling

Returning a view created from a local variable can create a dangling reference. This has always been the case and is inherent to `ranges::ref_view` (and independent of the underlying range). While it could be argued that preventing it for *some ranges* is beneficial, it is unclear why this should only be the case for move-only views and not other ranges; especially since example (2) is likely to appear more often than example (4) and the range in (3) is semantically identical to the range in (4).

On a sidenote: Now that we have move-only owning views, one could make an argument for making all uses of `ranges::ref_view` explicit (removing the option from `views::all` and recommending that users `move` into a view composition by default or explicitly opt into having a reference if desired). However, such a change would be quite disruptive, so pursuing this argument at this point seems futile.

§ 6.2. Returning views on in-parameters

Nr.	Code	Status quo	This proposal
(1)	<pre> auto foo(string_view & s) { auto v1 = s views::take(3); return v1 views::transform(**/); } </pre>	not dangling	not dangling
(2)	<pre> auto foo(string & s) { auto v1 = s views::take(3); return v1 views::transform(**/); } </pre>	not dangling	not dangling
(3)	<pre> auto foo(Generator & g) { auto v1 = g views::take(3); return v1 views::transform(**/); } </pre>	not dangling	not dangling
(4)	<pre> auto foo(GeneratorView & g) { auto v1 = g views::take(3); return v1 views::transform(**/); } </pre>	ill-formed	well-formed & not dangling

Example (4) would become well-formed code through the changes proposed here. Contrary to what one may assume, the example will not return a dangling reference: while `s` is move-only, `v1` will be a `ref_view` to `s` and thus copyable; in the return statement, it is copied into the returned view, so the composition does not dangle.

§ 7. Implication for the "meaning" of views

Objections to this proposal include that it would distort the meaning of views and that »[t]he entire point of a view is that this is something that should be taken by value and owned«. There is some irony here, because this criticism comes from the same people who removed all other defining features from views (default-constructibility, cheap copyability, not owning)! But I do not disagree with those changes and want to argue instead that (a) views are not *generally* taken by value; and (b) the view concept is not useful in most generic code anyway.

In fact, this argument is already made in [P2415](#):

The initial thought [...] might be that [...] we write algorithms that take views by value: [...] and we definitely do encourage people to take *specific* views by value - such as `span` and `string_view` [...]. Either way, we very much want range-based algorithms to be able to operate on, well, ranges, so these are always written instead to take ranges by forwarding reference:

```
template <input_range R>
void some_algo(R&& r);
```

→ we do not actually take views by value in generic code, and we do not use the view concept to constrain algorithms.

As far as the "meaning of a view" is concerned, the authors of [P1456](#) conclude a paragraph by writing:

the benefit that views provide: cheap construction of range adaptor pipelines.

I would argue that this is probably the only remaining useful definition of "view", and it is exactly what the changes proposed here try to make more consistent.

By strongly relaxing the constraints of `view` and `viewable_range`, [P1456](#) and [P2415](#) made it possible to use range adaptors without having to know how these concepts are defined or what they "mean". That's great, because most people do not need to know (and we have trouble explaining it ourselves 😊)!

There is, however, an exception to that rule where the "details" (what is a view? [when] is it copyable? why should I care?) leak out, and this paper fixes that.

§ 8. Summary

Treating move-only views as fundamentally different from copyable views when evaluating view-ability, creates a source of user error and frustration. I agree with the changes made to views, but we now have problems explaining what a view even is ([P2415](#)); having two sets of views with different usage patterns definitely won't help. This does not only affect exotic, user-defined types, but can become obvious even in toy examples created with strings.

The difference becomes even more pronounced with standalone move-only ranges that can be implemented as views but where we cannot assume that they always will be. For those types, it is unclear what benefit **opting into** the view concept provides—if that only makes the type **opt out** of easy view composition.

Further complications arise in generic code, which doesn't care about the view concept or ownership at all and could previously assume that all lvalue ranges are viewable. It now needs workarounds which are easy to forget and come with their own side-effects.

While it is true that all direct and indirect uses of `ranges::ref_view` entail the possibility of dangling references, it is unclear why this is considered more dangerous for move-only views than other ranges. I hope to have alleviated some concerns in this regard and shown that the proposed increase in consistency and usability provides a clear benefit.

§ 9. Acknowledgments

I want to thank Barry Revzin and Tim Song for their work on revising views and for their comments on the current topic. Thanks also to others for their feedback on the draft of this paper.

Thanks to my employer, deCODE Genetics, for sponsoring my WG21 time.

§ 10. References

- Short 2020 discussion in LWG: <https://lists.isocpp.org/lib/2020/09/17353.php>
- Short 2022 discussion in LWG: <https://lists.isocpp.org/lib/2022/08/23438.php>
- Brief mention in minutes: <https://wiki.edg.com/bin/view/Wg21telecons2021/LWG-issues-20210115>
- GCC issue with other examples: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=106669